



09 – OPERAÇÕES GEOMÉTRICAS

Bruno B. F. Leonor

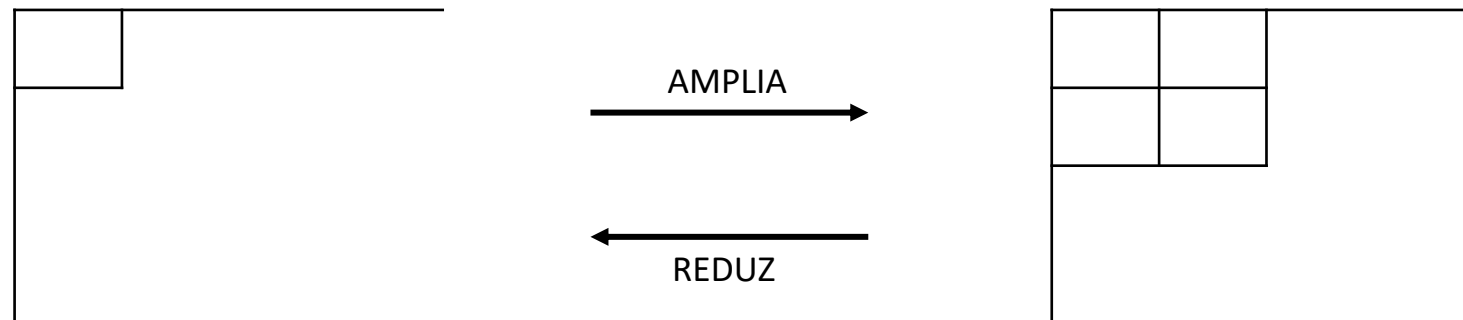
PROCESSAMENTO DIGITAL DE IMAGENS

Operações Geométricas

- Ampliação e Redução
- Extração de Regiões
- Métodos de Interpolação
 - Ordem-zero
 - Primeira-ordem
 - Mais alta ordem
- Transformação afim
 - Translação
 - Rotação
 - Mudança de escala
 - Distorção (shearing)
 - Concatenação
- Algoritmos de Transformação
- Classe AffineTransform

Ampliação e Redução

- São processos pelos quais uma imagem é aumentada ou reduzida de seu tamanho original.
- A maneira mais simples de ampliar é duplicar o valor do pixel por um valor inteiro de maneira igual nas direções X e Y.



Ampliação e Redução

- A resolução da imagem **NÃO** foi aumentada, apenas seu tamanho.
- Para a redução o processo não é tão direto – quando se reduz uma imagem, alguma informação tem de ser “perdida”.
- Abordagem adotada para reduzir perda de informação é usar quatro pixels vizinhos para produzir um contendo o valor médio.
- Para reduzir por um fator de 4, usam-se 8 pixels e assim sucessivamente.

Ampliação e Redução

- Essas técnicas podem ser usadas para aumentar pequenos detalhes de uma imagem, ou reduzir uma grande imagem para caber no vídeo, ou na área de visualização.
- Operação rápida – usar fator inteiro.
- Para valores fracionários – necessário algoritmos de interpolação.

Ampliação e Redução

```
1 package pdi;
2
3 // pacote genérico
4 import java.io.File;
5
6 // pacotes do AWT
7 import java.awt.image.BufferedImage;
8
9 // pacote Swing para interface gráfica
10 import javax.swing.JFrame;
11 import javax.swing.JLabel;
12 import javax.swing.ImageIcon;
13 import javax.swing.JScrollPane;
14
15 // leitura em modo imediato - para J2SE 1.4+
16 import javax.imageio.ImageIO;
17
18 public class AmpRed extends JFrame {
19
20     public static void main(String args[]) {...23 linhas }
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44     public AmpRed(String aFile, int aFator, int aTipoOp) {...25 linhas }
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70     public BufferedImage zoomImage(BufferedImage image, int fator, int tipoOp)
71
72
73
74
75
76 }
```

```
20
21
22     String img = "mini_einstein.jpg";
23
24     // tipoOp 1 - ampliação, 2 - redução
25     int fator = 1, tipoOp = 1;
26
27     // Assume valor padrão
28     if (fator <= 0)
29         fator = 1;
30     if ((tipoOp <= 0) || (tipoOp > 2))
31         tipoOp = 1;
32
33     // Mostra JFrame decorado pelo Swing
34     JFrame.setDefaultLookAndFeelDecorated(true);
35
36     // Chama o construtor
37     JFrame zoomImg = new AmpRed(img, fator, tipoOp);
38
39     // Encerra a aplicação clicando no "close"
40     zoomImg.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
41     zoomImg.setVisible(true);
42 }
```

Ampliação e Redução

```
44 public AmpRed(String aFile, int aFator, int aTipoOp) {  
45  
46     // Define um objeto BufferedImage para encapsular a imagem  
47     BufferedImage bimage = null, dest = null;  
48  
49     // Certifica que a imagem existe  
50     File file = new File(aFile);  
51     try {  
52         bimage = ImageIO.read(file);  
53     } catch (Exception e) {  
54         System.out.println("Imagem '" + aFile + "' nao existe.");  
55         System.exit(0);  
56     }  
57  
58     // Atribui nome ao frame  
59     setTitle("Amplia/Reduz imagem: " + file.getName());  
60  
61     // Aumenta/Reduz imagem pelo fator  
62     dest = zoomImage(bimage, aFator, aTipoOp);  
63  
64     // Define GUI com objetos do Swing  
65     JLabel limg = new JLabel(new ImageIcon(dest));  
66     getContentPane().add(new JScrollPane(limg));  
67     setSize(bimage.getWidth(), bimage.getHeight());  
68 }
```

Ampliação e Redução

```
70 public BufferedImage zoomImage(BufferedImage image, int fator, int tipoOp) {  
71     BufferedImage bi = null;  
72     int w, h;  
73  
74     switch(tipoOp) {  
75         // Amplia pelo fator  
76         case 1:  
77             w = fator*image.getWidth();  
78             h = fator*image.getHeight();  
79             bi = new BufferedImage(w, h, image.getType());  
80             for (int j = 0; j < h; j++)  
81                 for (int i = 0; i < w; i++)  
82                     bi.setRGB(i, j, image.getRGB(i/fator, j/fator));  
83             break;  
84         // Reduz pelo fator  
85         case 2:  
86             w = (int)image.getWidth()/fator;  
87             h = (int)image.getHeight()/fator;  
88             bi = new BufferedImage(w, h, image.getType());  
89             for (int j = 0; j < h; j++)  
90                 for (int i = 0; i < w; i++)  
91                     bi.setRGB(i, j, image.getRGB(i*fator, j*fator));  
92             break;  
93     }  
94     return bi;  
95 }
```

Extração de Regiões de Interesse (ROI)

- É uma área retangular dentro de uma imagem.
- Definida pelas coordenadas dos pixels dos cantos superior esquerdo e inferior direito ou canto superior esquerdo e suas dimensões.
- Utilizadas para limitar a área de operações de processamento.
- BufferedImage: Raster getData(Rectangle rect)
 void setData(Raster raster)
 BufferedImage getSubimage(int x, int x, int w, int h)

Extração de Regiões de Interesse (ROI)

- `getData(Rectangle rect)`
 - Parâmetro retorna os dados da ROI como um objeto Raster.
 - Dados são independentes da imagem, mudanças não afetarão a região.
 - Sistema de coordenadas da região é a mesma da imagem original.
- `setData(Raster raster)`
 - Raster é “*read-only*”.
 - `WritableRaster`
- `getSubimage(int x, int x, int w, int h)`
 - Processamento local na ROI.
 - Retorna a sub-imagem que compartilha a mesma área de dados que a imagem pai.
 - Mudanças afetarão a imagem pai.
 - Sistema de coordenadas não é o mesmo.

Extração de Regiões de Interesse (ROI)

```
1 package pdi;
2
3 // pacote genérico
4 import java.io.File;
5 import java.text.DecimalFormat;
6
7 // pacotes do AWT
8 import java.awt.image.BufferedImage;
9 import java.awt.image.Raster;
10 import java.awt.Rectangle;
11
12 // pacote Swing para interface gráfica
13 import javax.swing.JFrame;
14 import javax.swing.JLabel;
15 import javax.swing.ImageIcon;
16 import javax.swing.JScrollPane;
17
18 // leitura em modo imediato
19 import javax.imageio.ImageIO;
20
21 public class Regioes extends JFrame {
22
23     public static void main(String args[]) {...27 linhas }
24
25     public Regioes(String aFile, int x, int y, int w, int h) {...3
26
27     public double media(BufferedImage img, Rectangle REGIAO) {...1
28
29     public double media(BufferedImage src) {...9 linhas }
30
31 }
32
33 public static void main(String args[]) {
34     int x = 0, y = 0, // canto superior esquerdo da ROI
35     w = 20, h = 20; // altura e largura da ROI
36
37     String nomeImg = "tabuleiroNC.png";
38
39     // Assume valor padrão
40     if (x < 0)
41         x = 0;
42     if (y < 0)
43         y = 0;
44     if (x > w)
45         x = 0;
46     if (y > h)
47         y = 0;
48
49     // Mostra JFrame decorado pelo Swing
50     JFrame.setDefaultLookAndFeelDecorated(true);
51
52     // Chama o construtor
53     Regioes rm = new Regioes(nomeImg, x, y, w, h);
54
55     // Encerra a aplicacao clicando no "close"
56     rm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
57     rm.setSize(200, 200);
58     rm.setVisible(true);
59 }
```

Extração de Regiões de Interesse (ROI)

```
51 public Regioes(String aFile, int x, int y, int w, int h) {
52     // Define um objeto BufferedImage para encapsular a imagem
53     BufferedImage bimage = null;
54
55     // Certifica que a imagem existe
56     File file = new File(aFile);
57     try {
58         bimage = ImageIO.read(file);
59     } catch (Exception e) {
60         System.out.println("Imagem '" + aFile + "' nao existe.");
61         System.exit(0);
62     }
63
64     // Atribui nome ao frame
65     setTitle("Média da ROI: " + file.getName());
66
67     // Certifica se dimensões estão corretas
68     if (w > bimage.getWidth())
69         w = bimage.getWidth() - x;
70
71     if (h > bimage.getHeight())
72         h = bimage.getHeight() - y;
73
74     // Se w e h são maiores que imagem, calcula média de toda imagem
75     if ((w > bimage.getWidth()) && (h > bimage.getHeight())) {
76         w = bimage.getWidth();
77         h = bimage.getHeight();
78         x = 0;
79         y = 0;
80     }
81
82     // Chama função
83     Rectangle regioao = new Rectangle(x, y, w, h);
84     double mediaROI = Regioes.this.media(bimage, regioao);
85     DecimalFormat df = new DecimalFormat("0.000");
86     System.out.println("\nmedia " + df.format(mediaROI));
87 }
```

Extração de Regiões de Interesse (ROI)

```
89 public double media(BufferedImage img, Rectangle REGIAO) {
90
91     BufferedImage biRegiao = img.getSubimage(REGIAO.x, REGIAO.y,
92         REGIAO.width, REGIAO.height);
93
94     // Define GUI com objetos do Swing
95     JLabel limg = new JLabel(new ImageIcon(biRegiao));
96     getContentPane().add(new JScrollPane(limg));
97     setSize(biRegiao.getWidth(), biRegiao.getHeight());
98
99     // Chama outro construtor de meanValue com BufferedImage ROI
100     return media(biRegiao);
101 }
```

```
103 public double media(BufferedImage src) {
104     double sum = 0.0;
105
106     Raster raster = src.getRaster();
107     for (int i = 0; i < src.getHeight(); i++)
108         for (int j = 0; j < src.getWidth(); j++)
109             sum += raster.getSample(j, i, 0);
110     return sum / (src.getWidth() * src.getHeight());
111 }
```



esse (ROI)

```
1 package pdi;
2
3 // pacotes genéricos
4 import java.io.File;
5 import java.text.DecimalFormat;
6
7 // pacotes do AWT
8 import java.awt.event.*;
9 import java.awt.Rectangle;
10 import java.awt.image.Raster;
11 import java.awt.BorderLayout;
12 import java.awt.image.BufferedImage;
13
14 // pacote Swing para interface gráfica
15 import javax.swing.JFrame;
16 import javax.swing.JLabel;
17 import javax.swing.ImageIcon;
18 import javax.swing.JFileChooser;
19
20 import javax.imageio.ImageIO;
21
22 public class RegioesMouse extends JFrame {
23
24     int x1, y1, x2, y2;
25     JLabel labelCoordenadas = new JLabel();
26     String coordenadas = "";
27     boolean first = true;
28     BufferedImage imagem = null;
29
30     public static void main(String args[]) {...25 linhas }
31
32     public RegioesMouse(File file) {...52 linhas }
33
34     public double media(BufferedImage img, Rectangle REGIAO) {...8 linhas }
35
36     public double media(BufferedImage src) {...9 linhas }
37 }
```



Extração de Regiões de Interesse (ROI)

```
30 public static void main(String args[]) {  
31     File file = null;  
32     int result;  
33  
34     // Comandos Swing para abrir janela com arquivos.  
35     JFileChooser fileChooser = new JFileChooser(".");  
36     fileChooser.setDialogTitle("Abre imagem");  
37     result = fileChooser.showOpenDialog(null);  
38     if (result == JFileChooser.APPROVE_OPTION) {  
39         file = fileChooser.getSelectedFile();  
40     } else {  
41         System.out.println("Escolher imagem.");  
42         System.exit(0);  
43     }  
44  
45     // Mostra JFrame decorado pelo Swing  
46     JFrame.setDefaultLookAndFeelDecorated(true);  
47  
48     // Chama o construtor  
49     JFrame rm = new RegioesMouse(file);  
50  
51     // Encerra a aplicacao clicando no "close"  
52     rm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
53     rm.setVisible(true);  
54 }
```

```
108  
109 public double media(BufferedImage img, Rectangle REGIAO) {  
110  
111     BufferedImage biRegiao = img.getSubimage(REGIAO.x, REGIAO.y,  
112         REGIAO.width, REGIAO.height);  
113  
114     // Chama outro construtor de media com BufferedImage ROI  
115     return media(biRegiao);  
116 }  
117  
118 public double media(BufferedImage src) {  
119     double sum = 0.0;  
120  
121     Raster raster = src.getRaster();  
122     for (int i = 0; i < src.getHeight(); i++)  
123         for (int j = 0; j < src.getWidth(); j++)  
124             sum += raster.getSample(j, i, 0);  
125     return sum / (src.getWidth() * src.getHeight());  
126 }
```

Extração de Regiões de Interesse (ROI)

```
56 public RegioesMouse(File file) {
57
58     // Armazena arquivo imagem numa BufferedImage
59     try {
60         imagem = ImageIO.read(file);
61     } catch (Exception e) {
62         System.out.println("Imagem '" + file + "' nao existe.");
63         System.exit(0);
64     }
65
66     // Atribui nome ao frame
67     setTitle("Média da ROI: " + file.getName());
68
69     // Define GUI com objetos do Swing
70     JLabel labelImagem = new JLabel(new ImageIcon(imagem));
71     add(labelImagem, BorderLayout.NORTH);
72     setSize(imagem.getWidth(), imagem.getHeight());
73
74     // Adiciona evento do mouse: imprime coordenadas do ponto clicado
75     add(labelCoordenadas, BorderLayout.SOUTH);
76
77     MouseListener mlst = new MouseAdapter() {...29 linhas };
106
107     labelImagem.addMouseListener(mlst);
108 }
```

Extração de Regiões de Interesse (ROI)

```
77 MouseListener mlst = new MouseAdapter() {  
78     public void mouseClicked(MouseEvent evt) {  
79         int xmin, ymin, xmax, ymax, w, h;  
80         if (evt.getClickCount() == 1)  
81             coordenadas = evt.getX() + ", " + evt.getY();  
82         labelCoordenadas.setText(coordenadas);  
83         if (first) {  
84             x1 = evt.getX();  
85             y1 = evt.getY();  
86             first = false;  
87         } else {  
88             x2 = evt.getX();  
89             y2 = evt.getY();  
90  
91             // Chama função  
92             xmin = Math.min(x1, x2);  
93             ymin = Math.min(y1, y2);  
94             xmax = Math.max(x1, x2);  
95             ymax = Math.max(y1, y2);  
96             w = xmax - xmin;  
97             h = ymax - ymin;  
98             Rectangle roi = new Rectangle(xmin, ymin, w, h);  
99             double mediaROI = RegioesMouse.this.media(imagem, roi);  
100             DecimalFormat df = new DecimalFormat("0.000");  
101             System.out.println("Media ROI = " + df.format(mediaROI));  
102             first = true;  
103         }  
104     }  
105 };
```

Extração de Regiões de Interesse (ROI)

```
1 package pdi;
2
3 // pacotes genéricos
4 import ...24 linhas
5
28
29 public class RegioesPoligono extends JFrame {
30
31     // Vector - armazena os vértices do polígono lidos com o mouse
32     Vector pontos = new Vector();
33
34     // Label para imprimir as coordenadas dos vértices
35     JLabel labelCoordenadas = new JLabel();
36     String coordenadas = "";
37
38     // roi = identificador do número da região.
39     // Usado para identificar o arquivo de saída.
40     int roi = 1;
41     BufferedImage imagem = null, copiaImagem = null;
42
43     public static void main(String args[]) {...29 linhas }
44
45
46     public RegioesPoligono(final File file) {...46 linhas }
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120 // MÉTODOS PRIVADOS
121 private void desenhaROI() {...28 linhas }
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149 private void fechaROI() {...27 linhas }
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178 private void meanValue(File file) {...84 linhas }
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262 }
```

```
43 public static void main(String args[]) {
44
45     File file = null;
46     int result;
47
48     // Comandos Swing para abrir janela com arquivos.
49     JFileChooser fileChooser = new JFileChooser(".");
50     fileChooser.setDialogTitle("Abre imagem");
51     result = fileChooser.showOpenDialog(null);
52     if (result == JFileChooser.APPROVE_OPTION) {
53         file = fileChooser.getSelectedFile();
54     } else {
55         System.out.println("Escolher imagem.");
56         System.exit(0);
57     }
58
59     // Mostra JFrame decorado pelo Swing
60     JFrame.setDefaultLookAndFeelDecorated(true);
61
62     // Chama o construtor
63     JFrame rm = new RegioesPoligono(file);
64
65     // Encerra a aplicação clicando no "close"
66     rm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
67     rm.setVisible(true);
68
69     // Executa 'garbage collector' para liberar recursos usados.
70     System.gc();
71 }
```

Extração de Regiões de Interesse (ROI)

```
73 public RegioesPoligono(final File file) {
74
75     // Armazena arquivo imagem numa BufferedImage
76     try {
77         imagem = ImageIO.read(file);
78     } catch (Exception e) {
79         System.out.println("Imagem '" + file + "' nao existe.");
80         System.exit(0);
81     }
82
83     // Copia imagem fonte
84     WritableRaster destWR = imagem.copyData(null);
85     copiaImagem = new BufferedImage(imagem.getColorModel(),
86                                     destWR, false, null);
87
88     // Atribui nome ao frame
89     setTitle("Média da ROI: " + file.getName());
90
91     // Define GUI com objetos do Swing
92     JLabel labelImagem = new JLabel(new ImageIcon(copiaImagem));
93     getContentPane().add(labelImagem, BorderLayout.NORTH);
94     setSize(imagem.getWidth(), imagem.getHeight());
95
96     // Adiciona evento do mouse: mostra coordenadas do ponto clicado
97     getContentPane().add(labelCoordenadas, BorderLayout.SOUTH);
98
99     MouseListener mlst = new MouseAdapter() {...18 linhas };
117     labelImagem.addMouseListener(mlst);
118 }
```

```
99 MouseListener mlst = new MouseAdapter() {
100     public void mouseClicked(MouseEvent evt) {
101         if (evt.getClickCount() == 1) {
102             coordenadas = evt.getX() + ", " + evt.getY();
103             labelCoordenadas.setText(coordenadas);
104             pontos.addElement(new Point(evt.getX(), evt.getY()));
105             desenhaROI();
106         }
107         if (evt.getClickCount() == 2) {
108             fechaROI();
109             meanValue(file);
110             // Libera área do Vector pontos
111             pontos.clear();
112             // Incrementa o identificador da ROI
113             roi++;
114         }
115     }
116 };
```

Extração de Regiões de Interesse (ROI)

```
120 // MÉTODOS PRIVADOS
121 private void desenhaROI() {
122     Point pt;
123     Ellipse2D ellipse;
124     GeneralPath path = new GeneralPath();
125
126     // Define contexto gráfico na imagem cópia (copiaImagem)
127     // para desenhar os vértices e segmentos de linhas.
128     Graphics2D g2 = copiaImagem.createGraphics();
129     g2.setColor(Color.red);
130
131     // Acessa primeiro vértice
132     pt = (Point) pontos.get(0);
133
134     // Move para o primeiro vértice
135     path.moveTo((float) pt.x, (float) pt.y);
136     for (int i = 0; i < pontos.size(); i++){
137         pt = (Point) pontos.get(i);
138         // Define círculo de raio 3 nos vértices
139         ellipse = new Ellipse2D.Float(pt.x, pt.y, 3, 3);
140         // Desenha os círculos preenchidos
141         g2.fill(ellipse);
142         // Desenha segmento de reta entre dois vértices
143         path.lineTo((float) pt.x, (float) pt.y);
144     }
145     // Desenha o polígono
146     g2.draw(path);
147     repaint();
148 }
```

```
150 private void fechaROI() {
151     Point pt;
152     GeneralPath path = new GeneralPath();
153     Ellipse2D ellipse;
154
155     // Define contexto gráfico na imagem cópia (copiaImagem)
156     // para desenhar os vértices e segmentos de linhas.
157     Graphics2D g2 = copiaImagem.createGraphics();
158
159     g2.setColor(Color.red);
160
161     // Acessa primeiro vértice
162     pt = (Point) pontos.get(0);
163
164     // Move para o primeiro vértice
165     path.moveTo((float) pt.x, (float) pt.y);
166     pt = (Point) pontos.get(pontos.size()-1);
167     // Define círculo de raio 3 nos vértices
168     ellipse = new Ellipse2D.Float(pt.x, pt.y, 3, 3);
169     // Desenha os círculos preenchidos
170     g2.fill(ellipse);
171     // Desenha segmento de reta entre dois vértices
172     path.lineTo((float) pt.x, (float) pt.y);
173     // Desenha o polígono
174     g2.draw(path);
175     repaint();
176 }
```

teresse (ROI)

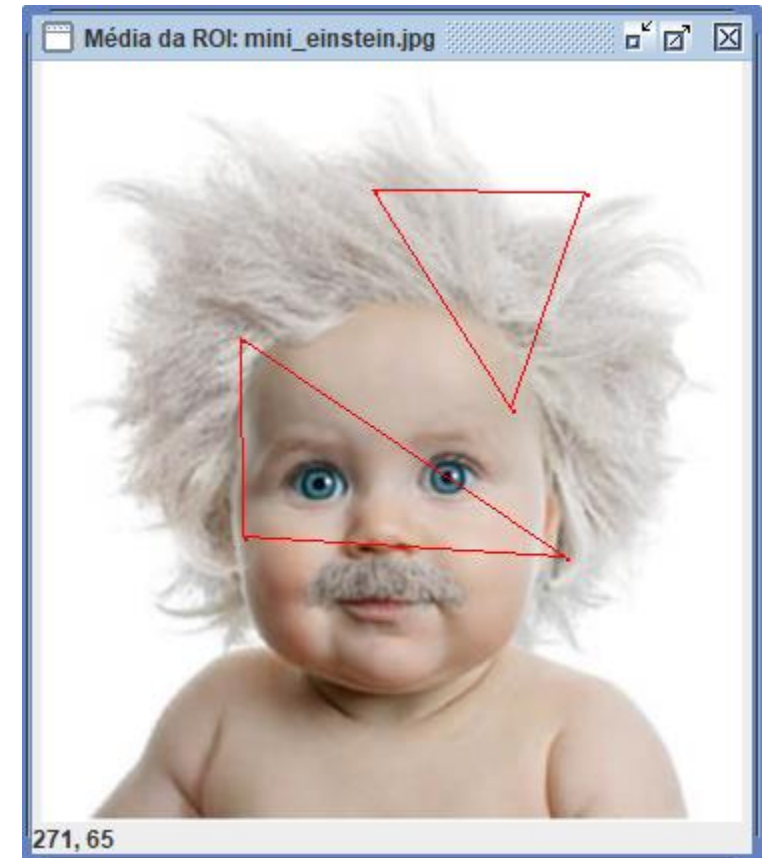
```
178 private void meanValue(File file) {
179
180     double sum = 0.0, media = 0.0;
181     int conta = 0;
182     int xmin, xmax, ymin, ymax, tipo, pixel;
183
184     // Prepara imagem para leitura de pixels
185     Raster raster = imagem.getRaster();
186
187     // Define figura geométrica genérica ou polígono
188     GeneralPath gp = new GeneralPath(GeneralPath.WIND_EVEN_ODD);
189
190     // Desenha polígono - cria contexto gráfico
191     Graphics2D g2 = copiaImagem.createGraphics();
192
193     // 1. move para ponto inicial
194     Point p = new Point();
195     p = (Point)pontos.get(0);
196     gp.moveTo((float)p.x, (float)p.y);
197
198     // 2. desenha os segmentos de reta
199     for (int i = 1; i < pontos.size(); i++) {
200         p = (Point)pontos.get(i);
201         gp.lineTo((float)p.x, (float)p.y);
202     }
203
204     // 3. fecha o polígono
205     gp.closePath();
206
207     // Acha máximos e mínimos para definir retângulo
208     // envolvente do polígono
209     p = (Point)pontos.get(0);
210     xmin = p.x; xmax = p.x;
211     ymin = p.y; ymax = p.y;
```

Extração de Regiões de Interesse (ROI)

```
212
213     for (int i = 1; i < pontos.size(); i++) {
214         p = (Point)pontos.get(i);
215         xmin = (int)Math.min(xmin, p.x);
216         ymin = (int)Math.min(ymin, p.y);
217         xmax = (int)Math.max(xmax, p.x);
218         ymax = (int)Math.max(ymax, p.y);
219     }
220
221     // Prepara imagem para gravar polígono
222     tipo = BufferedImage.TYPE_BYTE_GRAY;
223     BufferedImage dest = new BufferedImage((xmax-xmin), (ymax-ymin), tipo);
224     WritableRaster destWR = dest.getRaster();
225
226     // Percorre todo o retângulo envolvente do polígono;
227     // se o ponto estiver dentro do polígono, lê valor do
228     // pixel e acumula soma
229     for (int i = ymin; i < ymax; i++)
230         for (int j = xmin; j < xmax; j++) {
231             if (gp.contains((double)j, (double)i)) {
232                 pixel = raster.getSample(j, i, 0);
233                 sum += pixel;
234                 conta++;
235                 destWR.setSample(j-xmin, i-ymin, 0, pixel);
236             }
237         }
238
239     media = sum/conta;
240     DecimalFormat df = new DecimalFormat("0.000");
241     System.out.println("Media ROI = " + df.format(media));
```

Extração de Regiões de Interesse (ROI)

```
242
243 // Salva a imagem no formato "png", que preserva os
244 // valores exatos acima. O formato "jpg" acrescenta
245 // efeitos as cores (valores) acima definidas.
246 /* Se você quiser gravar o arquivo, modificar o diretório
247 String saida = null;
248 try {
249     // Recupera o nome da imagem, sem extensao
250     String nomeImg = file.getName();
251     String search = ".";
252     int i = nomeImg.indexOf(search);
253     saida = nomeImg.substring(0, i);
254     saida += "_ROI_" + String.valueOf(roi) + ".png";
255     System.out.println("Criando imagem '" + saida);
256     ImageIO.write(dest, "png", new File(saida));
257 } catch (IOException e) {
258     System.out.println("Problema gravando arquivo: " + saida);
259     System.exit(0);
260 }
261 */
262 }
263 }
```



Interpolação

- Usados sempre que se deve calcular valores de pixels em posições diferentes da original, como em processos de rotação de imagem.
- É um processo de gerar um novo valor de pixel com base nos valores de seus vizinhos.
- Os pixels vizinhos contribuem com pesos diferentes para o cálculo do valor do pixel.
- Java2D oferece suporte para técnicas de interpolação.

Transformação afim

- Transformação é uma função que mapeia um ponto, de um lugar no espaço para outro.
- Transformação afim preserva certas propriedades após sua execução.
 - translação
 - mudança de escala
 - rotação
- Quando aplicadas, elas não distorcem o objeto, pois o paralelismo e ângulos internos são todos preservados – transformação de corpo rígido.

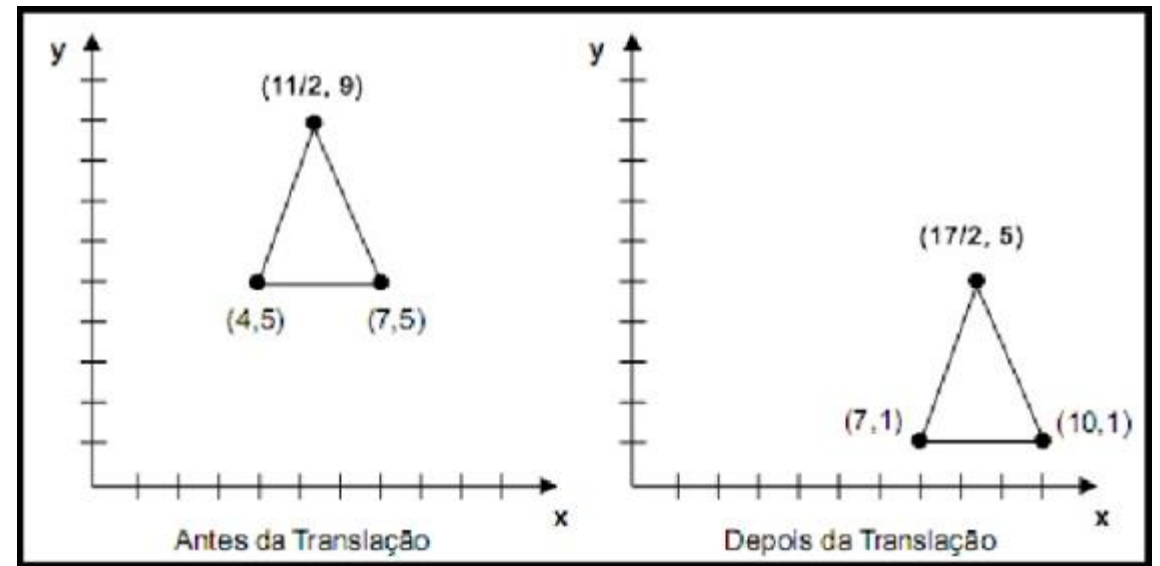
Transformação de Translação

- Significa movimentar o objeto.
- Desloca o objeto por uma distância fixa nas direções X, Y e Z.

$$x' = x + Tx$$

$$y' = y + Ty$$

$$z' = z + Tz$$

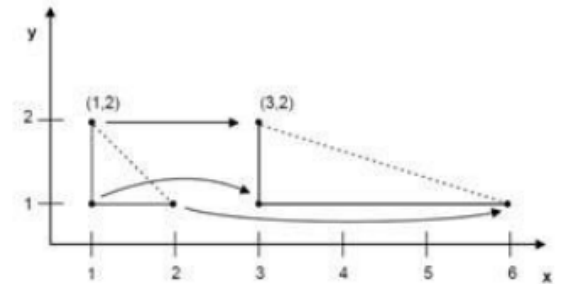
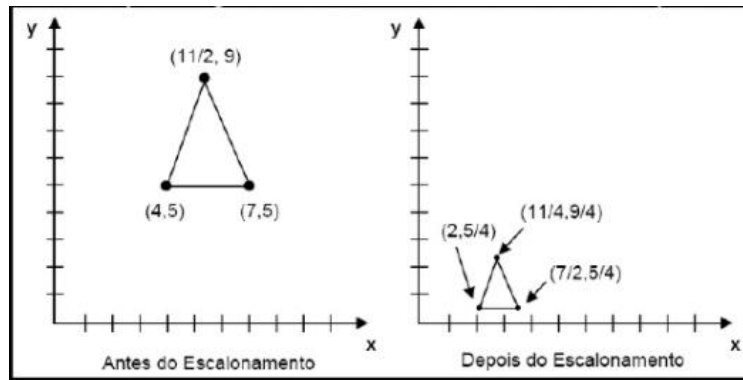


$$P' = P + T = [x' y' z'] = [x y z] + [Tx Ty Tz]$$

Transformação de Escala

- Significa mudar as dimensões.
- Para mudar o tamanho do objeto, teremos de multiplicar os valores de suas coordenadas por um fator de escala.

$$\begin{aligned}x' &= x * S_x \\ y' &= y * S_y \\ z' &= z * S_z\end{aligned}$$



- Se o objeto não estiver definido em relação à origem, essa operação de multiplicação também fará com que o objeto também translate.
- Transf. de escala não é uma alteração de corpo rígido já que geralmente deforma o corpo transformado.

Transformação de Rotação

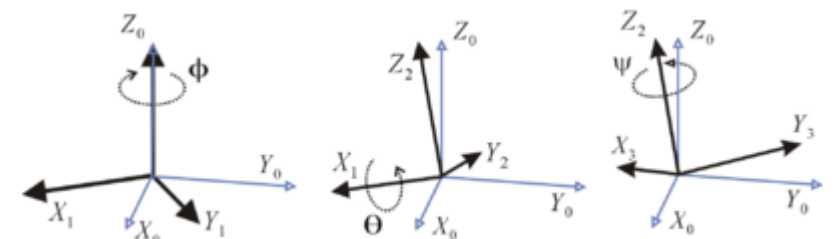
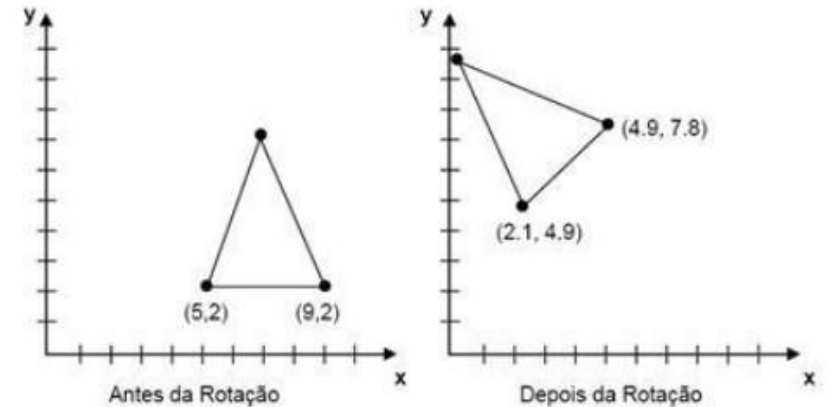
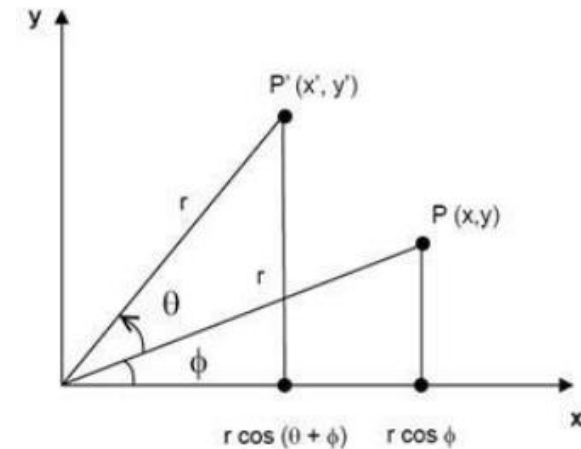
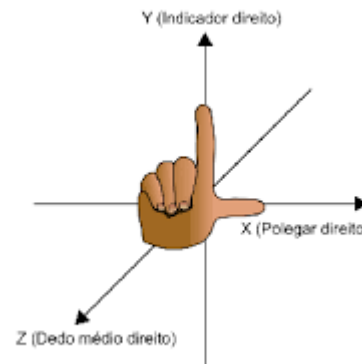
- Significa girar

$$x' = x * \cos\theta - y * \sin\theta$$

$$y' = y * \cos\theta + x * \sin\theta$$

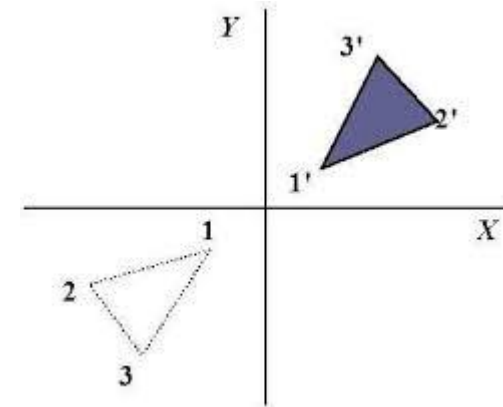
- A multiplicação das coordenadas por uma matriz de rotação pode resultar em uma translação.

- É possível rotacionar em qualquer plano (xy, yz, xz)

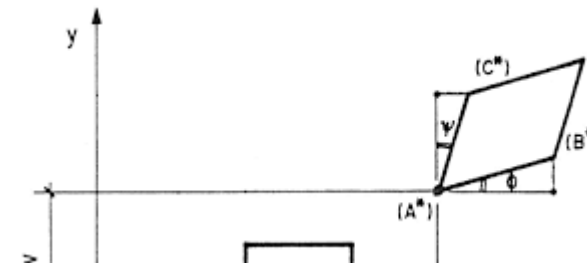


Transformação de Distorção (Shearing)

- Reflexão – produz um efeito de espelho no objeto, como se ele fosse virado em torno do eixo.



- Distorção – modifica a forma do objeto de maneira que ele pareça alongado na direção da distorção.



Concatenação

- Em muitos problemas reais é necessário aplicar transformações afins em primitivas em sequência.
- Aplicar uma a uma requer muitas operações computacionais.
- O mesmo resultado pode ser obtido concatenando todas as transformações e aplicando apenas uma vez.
- As transformações elementares podem ser compostas, como um processo de empilhamento.

Algoritmos de transformação

- Mapeamento indutivo – envolve uma iteração sobre cada pixel da imagem de entrada calculando novas coordenadas e copiando seu valor para uma nova posição.
 - As novas coordenadas podem não ser mapeadas na imagem de saída por não atenderem o requisito da grade de números inteiros positivos.
- Mapeamento dedutivo – garante um valor para todos os pixels da imagem de saída usando a transformação inversa para determinar a posição na imagem de entrada, da qual o valor do pixel deve ser copiado.

Algoritmos de transformação

MAPEAMENTO INDUTIVO

criar s (img_saída) $M \times N$

para todos os pixels em s fazer:

$$s(x,y)=0$$

$$a_0 = \cos\theta$$

$$b_0 = \sin\theta$$

$$a_1 = -b_0$$

$$b_1 = a_0$$

para todos os pixels de e fazer:

$$x' = \text{round}(a_0x + a_1y)$$

$$y' = \text{round}(a_1x + b_1y)$$

se (x',y') estiver em s então

$$s(x',y') = e(x,y)$$

MAPEAMENTO DEDUTIVO

criar s (img_saída) $M \times N$

criar e (img_entrada) $M \times N$

$$a_0 = \cos\theta$$

$$a_1 = \sin\theta$$

$$b_0 = -a_1$$

$$b_1 = a_0$$

para todos os pixels de s fazer:

$$x = \text{round}(a_0x' + a_1y')$$

$$y = \text{round}(b_0x' + b_1y')$$

se (x,y) estiver em e então

$$s(x,y) = e(x,y)$$

senão

$$s(x,y)=0$$

Mapeamento Indutivo

```
1 package pdi;
2
3 // pacotes genéricos
4 import java.io.File;
5 import java.text.DecimalFormat;
6
7 // pacotes do AWT
8 import java.awt.image.Raster;
9 import java.awt.image.BufferedImage;
10 import java.awt.image.WritableRaster;
11
12 // pacote Swing para interface gráfica
13 import javax.swing.JFrame;
14 import javax.swing.JLabel;
15 import javax.swing.ImageIcon;
16 import javax.swing.JScrollPane;
17
18 // leitura em modo imediato - para J2SE 1.4+
19 import javax.imageio.ImageIO;
20
21 public class RotacaoIndutiva extends JFrame {
22
23     public static void main(String args[]) {...21 linhas }
24
25     public RotacaoIndutiva(String aFile, double angulo) {...28 linhas }
26
27     public BufferedImage rotacao(BufferedImage src, double theta) {...34 linhas }
28 }
```

```
23 public static void main(String args[]) {
24     double theta = 15.0; // angulo de rotação
25     String nomeImg = "mini_einstein.jpg";
26
27     // Tempo atual
28     long e_time = System.currentTimeMillis();
29
30     // Mostra JFrame decorado pelo Swing
31     JFrame.setDefaultLookAndFeelDecorated(true);
32
33     // Chama o construtor passando parâmetros
34     JFrame ri = new RotacaoIndutiva(nomeImg, theta);
35
36     // Encerra a aplicação clicando no "close"
37     ri.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38     ri.setVisible(true);
39
40     e_time = System.currentTimeMillis() - e_time;
41     DecimalFormat df = new DecimalFormat("0.0000");
42     System.out.println("Tempo de execucao: "
43         + df.format(e_time) + " milisseg.");
44 }
```

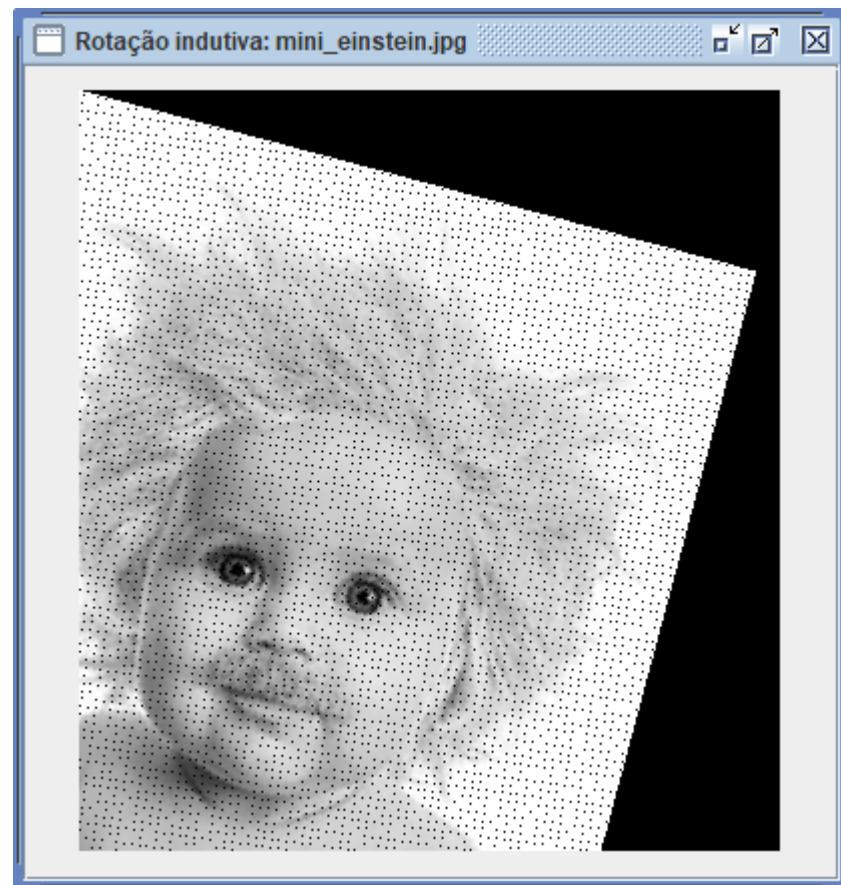
Mapeamento Indutivo

```
46 public RotacaoIndutiva(String aFile, double angulo) {
47     // Define um objeto BufferedImage para encapsular a imagem
48     BufferedImage src = null, dest = null;
49     JLabel destL;
50     int w, h;
51
52     // Certifica que a imagem existe
53     File file = new File(aFile);
54     try {
55         src = ImageIO.read(file);
56     } catch (Exception e) {
57         System.out.println("Imagem '" + aFile + "' nao existe.");
58         System.exit(0);
59     }
60
61     // Atribui nome ao frame
62     setTitle("Rotação indutiva: " + file.getName());
63
64     // Realiza operação de rotação na imagem
65     dest = rotacao(src, angulo);
66
67     w = dest.getWidth();
68     h = dest.getHeight();
69
70     destL = new JLabel(new ImageIcon(dest));
71     getContentPane().add(new JScrollPane(destL));
72     setSize(w, h);
73 }
```

```

75 public BufferedImage rotacao(BufferedImage src, double theta) {
76     // Para imagens com níveis de cinza
77     int w, h, tipo;
78
79     w = src.getWidth();
80     h = src.getHeight();
81     tipo = BufferedImage.TYPE_BYTE_GRAY;
82     BufferedImage out = new BufferedImage(w, h, tipo);
83
84     Raster srcR = src.getRaster();
85     WritableRaster outWR = out.getRaster();
86
87     double a0 = Math.cos(theta*Math.PI/180.0);
88     double b0 = Math.sin(theta*Math.PI/180.0);
89     double a1 = -b0,
90     b1 = a0;
91
92     int rx, ry;
93     for (int y = 0; y < h; y++)
94         for (int x = 0; x < w; x++) {
95             rx = (int) Math.round(a0*x + a1*y);
96             ry = (int) Math.round(b0*x + b1*y);
97             if (rx >= 0 && rx < w && ry >= 0 && ry < h) {
98                 try {
99                     outWR.setSample(rx, ry, 0, srcR.getSample(x, y, 0));
100                 } catch (ArrayIndexOutOfBoundsException e) {
101                     String str = "Problema de indice na imagem de saida : ";
102                     System.out.println(str + y + ", " + x);
103                     System.exit(0);
104                 }
105             }
106         }
107     return out;
108 }

```



Mapeamento Dedutivo

```
1 package pdi;
2
3 // pacotes genéricos
4 + import ...16 linhas
20
21 public class RotacaoIndutiva extends JFrame {
22
23 + public static void main(String args[]) {...22 linhas }
45
46 + public RotacaoIndutiva(String aFile, double angulo) {...28 linhas }
74
75 + public BufferedImage rotacao(BufferedImage src, double theta) {...34
109 }
```

```
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44

public static void main(String args[]) {
    double theta = 15.0; // angulo de rotação
    String nomeImg = "mini_einstein.jpg";

    // Tempo atual
    long e_time = System.currentTimeMillis();

    // Mostra JFrame decorado pelo Swing
    JFrame.setDefaultLookAndFeelDecorated(true);

    // Chama o construtor passando parâmetros
    JFrame ri = new RotacaoIndutiva(nomeImg, theta);

    // Encerra a aplicação clicando no "close"
    ri.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    ri.setVisible(true);

    e_time = System.currentTimeMillis() - e_time;
    DecimalFormat df = new DecimalFormat("0.0000");
    System.out.println("Tempo de execucao: "
        + df.format(e_time) + " milisseg.");
}
```

Mapeamento Dedutivo

```
46 public RotacaoDedutiva(String aFile, double angulo) {
47     // Define um objeto BufferedImage para encapsular a imagem
48     BufferedImage src = null, dest = null;
49     JLabel destL;
50     int w, h;
51
52     // Certifica que a imagem existe
53     File file = new File(aFile);
54     try {
55         src = ImageIO.read(file);
56     } catch (Exception e) {
57         System.out.println("Imagem " + aFile + " nao existe.");
58         System.exit(0);
59     }
60
61     // Atribui nome ao frame
62     setTitle("Rotação dedutiva: " + file.getName());
63
64     // Realiza operação de rotação na imagem
65     dest = rotacao(src, angulo);
66
67     w = dest.getWidth();
68     h = dest.getHeight();
69
70     destL = new JLabel(new ImageIcon(dest));
71     getContentPane().add(new JScrollPane(destL));
72     setSize(w, h);
73 }
```

```

75 public BufferedImage rotacao(BufferedImage src, double theta) {
76
77     // Para imagens com níveis de cinza
78     int w, h, tipo;
79
80     w = src.getWidth();
81     h = src.getHeight();
82     tipo = BufferedImage.TYPE_BYTE_GRAY;
83     BufferedImage out = new BufferedImage(w, h, tipo);
84
85     Raster srcR = src.getRaster();
86     WritableRaster outWR = out.getRaster();
87
88     double a0 = Math.cos(theta*Math.PI/180.0);
89     double a1 = Math.sin(theta*Math.PI/180.0);
90     double b0 = -a1,
91     b1 = a0;
92
93     int x, y;
94     for (int ry = 0; ry < h; ry++)
95         for (int rx = 0; rx < w; rx++) {
96             x = (int) Math.round(a0*rx + a1*ry);
97             y = (int) Math.round(b0*rx + b1*ry);
98             if (x >= 0 && x < w && y >= 0 && y < h) {
99                 try {
100                     outWR.setSample(rx, ry, 0, srcR.getSample(x, y, 0));
101                 } catch (ArrayIndexOutOfBoundsException e) {
102                     String str = "Problema de indice na imagem de saida : ";
103                     System.out.println(str + y + ", " + x);
104                     System.exit(0);
105                 }
106             } else {

```

Mapeamento Dedutivo

```
107 | | | | | try {
108 | | | | |     outWR.setSample(rx, ry, 0, 0);
109 | | | | | } catch (ArrayIndexOutOfBoundsException e) {
110 | | | | |     String str = "Problema de indice na imagem de saida : ";
111 | | | | |     System.out.println(str + y + ", " + x);
112 | | | | |     System.exit(0);
113 | | | | | }
114 | | | | | }
115 | | | | | }
116 | | | | | return out;
117 | | | | | }
```

